

APRENDIZAJE AUTOMÁTICO

Trabajo Práctico #1



Por

Franco Orellana
Thiago García
Máxima Pellerano



EJERCICIO 1

DIVISIÓN DE DATOS

Usamos la librería **PANDAS** para separar el dataset en los primeros ejemplos que cubren el 75 % del total de ejemplos como conjunto de **entrenamiento** y el resto como conjunto de **prueba**.

```
# Cargar los datos
datos = pd.read_csv("Préstamo.csv")

datos_originales = datos.copy()

if "Edad" in datos.columns:
    datos = datos[datos["Edad"] == 50].reset_index(drop=True)

# 1. DIVISIÓN DEL CONJUNTO (75% Entrenamiento / 25% Prueba)
n_entrenamiento = int(len(datos) * 0.75)

entrenamiento = datos.iloc[:n_entrenamiento].copy()
testeo = datos.iloc[n_entrenamiento:].copy()
```

ALGORITMO FIND-S

Comienza con la hipótesis más específica y la generaliza considerando sólo los ejemplos positivos

Empezamos con la hipótesis más específica:

$$h = [\emptyset, \emptyset, \dots, \emptyset]$$

Terminamos con una hipótesis mas general:

$$h = [?, x, x, \dots, ?]$$

- **Ignora los ejemplos negativos, solo tiene en cuenta los positivos**
- **Encuentra la hipótesis más específica que es consistente con los ejemplos positivos**
- **En este caso, buscamos obtener la hipótesis mas general para las personas de 50 años cuyo estado de solicitud es OTORGADO (+)**

```
# Función FIND-S
```

```
def find_s(datos_entrenamiento):
```

```
    # La hipótesis comienza siendo totalmente específica
```

```
    hipotesis = None
```

```
    for _, ejemplo in datos_entrenamiento.iterrows():
```

```
        if ejemplo["Estado"] == "OTORGADO":
```

```
            valores = [ejemplo[atributo] for atributo in atributos]
```

```
            # Primer ejemplo positivo
```

```
            hipotesis = inicializar_hipotesis(hipotesis, valores)
```

```
    return hipotesis
```

```
def inicializar_hipotesis(hipotesis, valores):
```

```
    if hipotesis is None:
```

```
        hipotesis = valores
```

```
    else:
```

```
        comparar_con_hipotesis_actual(hipotesis, valores)
```

```
    return hipotesis
```

```
def comparar_con_hipotesis_actual(hipotesis, valores):
```

```
    # Comparar con la hipótesis actual
```

```
    for i in range(len(atributos)):
```

```
        if hipotesis[i] != valores[i]:
```

```
            hipotesis[i] = "?"
```

PREDICCIONES

```
def realizar_predicciones():  
    hipotesis = find_s(entrenamiento)  
  
    testeo["Prediccion"] = testeo.apply(  
        lambda fila: predecir(fila, hipotesis),  
        axis=1  
    )  
  
    print("\nPredicciones:")  
    print(testeo[atributos + ["Estado", "Prediccion"]])
```

Usamos la hipotesis obtenida con FIND-S para predecir si a una persona se le otorga o se le rechaza.

CODE REVIEW

#1

EJERCICIO 2

MATRIZ DE CONFUSIÓN

Relaciona los datos positivos y negativos predichos con los datos reales

VALORES PREDICCIÓN	Verdaderos positivos	Falsos Positivos
	Falsos Negativos	Verdaderos Negativos
VALORES REALES		

Nos permite saber como clasifico nuestro modelo los distintos ejemplos del set de pruebas

IMPLEMENTACIÓN DE MATRIZ DE CONFUSIÓN

```
def matriz_confusion(y_real, y_pred):  
    VP = 0  
    VN = 0  
    FP = 0  
    FN = 0  
    for real, prediccion in zip(y_real, y_pred):  
        if real == "OTORGADO" and prediccion == "OTORGADO":  
            VP += 1  
        elif real == "RECHAZADO" and prediccion == "RECHAZADO":  
            VN += 1  
        elif real == "RECHAZADO" and prediccion == "OTORGADO":  
            FP += 1  
        elif real == "OTORGADO" and prediccion == "RECHAZADO":  
            FN += 1  
  
    return VP, VN, FP, FN
```

Matriz de Confusión:

Verdaderos Positivos (TP): 4

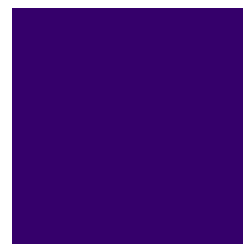
Verdaderos Negativos (TN): 6

Falsos Positivos (FP): 4

Falsos Negativos (FN): 0

MÉTRICAS DE EVALUACIÓN

Podemos usar los datos de la matriz de confusión para calcular distintas metricas.



ACCURACY
 $(TP + TN) / TOTAL$



RECALL
 TP / P



SPECIFICITY
 TN / N



PRECISION
 $TP / TP + FP$



TRUE POSITIVE RATE
 TP / P



FALSE POSITIVE RATE
 FP / N



F1-SCORE
 $2 * precision * recall / precision + recall$



IMPLEMENTACIÓN DE MÉTRICAS DE EVALUACIÓN

```
def calcular_metricas(VP, VN, FP, FN):  
  
    accuracy = (VP + VN) / (VP + VN + FP + FN)  
    recall = VP / (VP + FN)  
    especificidad = VN / (VN + FP)  
    precision = VP / (VP + FP)  
  
    print("\nMétricas:")  
    print(f"Accuracy: {accuracy:.4f}")  
    print(f"Recall: {recall:.4f}")  
    print(f"Especificidad: {especificidad:.4f}")  
    print(f"Precisión: {precision:.4f}")  
  
    return accuracy, recall, especificidad, precision
```

IMPLEMENTACIÓN DE MÉTRICAS DE EVALUACIÓN

```
def calcular_tasas(VP, VN, FP, FN):  
    TPR = VP / (VP + FN)  
    FPR = FP / (FP + VN)  
    print(f"\nTasa de verdaderos positivos (TPR): {TPR:.4f}")  
    print(f"Tasa de falsos positivos (FPR): {FPR:.4f}")  
    return TPR, FPR
```

```
def calcular_f1(precision, recall):  
    f1 = 2 * (precision * recall) / (precision + recall)  
    print(f"\nF1-score: {f1:.4f}")  
    return f1
```

CODE REVIEW

#2

EJERCICIO 3

DIVISIÓN DE DATOS

Usamos la librería **PANDAS** para separar el dataset aleatoriamente, dejando el 80 % del total de ejemplos como conjunto de **entrenamiento** y el 20% restante como conjunto de **prueba**.

```
# Cargar datos del csv
data = pd.read_csv('Préstamo.csv')
data.columns = data.columns.str.strip()

# Filtrar para personas de entre 40 y 45 años
data_filtrada = data[(data['Edad'] >= 40) & (data['Edad'] <= 45)].copy()

atributos = ['Sexo', 'Mayor nivel educativo', 'Estado de vivienda', 'Préstamos previos impagos']
atributo_concepto = 'Estado'

PORCENTAJE_ENTRENAMIENTO = 0.80

# Mezclar y extraer el 80% de los datos para entrenamiento de forma aleatoria
set_entrenamiento = data_filtrada.sample(frac=PORCENTAJE_ENTRENAMIENTO, random_state=42)

# El set de prueba es el resto del dataset (20%)
set_prueba = data_filtrada.drop(set_entrenamiento.index)
```

NAIVE BAYES

Vamos a usar la técnica de Naive Bayes para clasificar los ejemplos del conjunto de prueba, el cual toma como base el Teorema de Bayes.

- Se llama “naive” (ingenuo) porque asumimos el supuesto independencia entre los atributos de nuestros ejemplos.
- Vamos a hacer una clasificación probabilística para que el modelo pueda decidir en base a su entrenamiento una clasificación para nuevos ejemplos.



ESTRUCTURAR EL PROBLEMA

Primero tuvimos que adaptar nuestros datos al modelo.

- La salida (Y): Es la clase a predecir
 - (OTORGADO o RECHAZADO)
- Los atributos (Xj): Son las características independientes de nuestros ejemplos
 - ('Sexo', 'Mayor nivel educativo', 'Estado de vivienda', etc)

También definimos otros datos que van a ser importantes más adelante:

Datos usados para entrenamiento de Naive Bayes:

Total de ejemplos: $N = 1141$

Cantidad de atributos: $d = 4$

Cantidad de clases: $K = 2$ (OTORGADO, RECHAZADO)

Cantidad de posibles valores de los atributos:

$M_0 = 2$ (Sexo) $\rightarrow$ ['FEMENINO', 'MASCULINO']

$M_1 = 5$ (Mayor nivel educativo) $\rightarrow$ ['UNIVERSITARIO', 'MAESTRÍA', 'SECUNDARIO', 'TERCIARIO', 'DOCTORADO']

$M_2 = 4$ (Estado de vivienda) $\rightarrow$ ['HIPOTECA', 'INQUILINO', 'PROPIETARIO', 'OTRO']

$M_3 = 2$ (Préstamos previos impagos) $\rightarrow$ ['SI', 'NO']

PROBABILIDADES A PRIORI (π_k)

Probabilidad de que ocurra una clase antes de ver los atributos.
Basicamente que tan frecuente es una clase en nuestro set de datos inicial.

P (OTORGADO) = Cantidad de personas con prestamo otorgado / Cantidad total de personas

P (RECHAZADO) = Cantidad de personas con prestamo rechazado / Cantidad total de personas

$$\hat{\pi}_k = P(Y = C_k) = \frac{\#\{Y = C_k\}}{N} \quad (\text{Más adelante tuvimos que usar un suavizado de Laplace dentro del cálculo})$$

Probabilidades a priori:

$$P(\text{OTORGADO}) = 236 / 1141 = 0.21$$

$$P(\text{RECHAZADO}) = 905 / 1141 = 0.79$$

VEROSIMILITUDES

Para cada atributo X_j calculamos qué tan probable es observar un valor específico X_{jm} cuando ya sabemos que el ejemplo pertenece a una clase determinada C_k

Basicamente, contar cuantas veces aparece un valor específico en los ejemplos de una clase en concreto, además de suavizar el cálculo con el **suavizado de Laplace** ($l = 1$) para evitar multiplicar por un posible 0 a la hora de hacer la predicción.

```
[ CLASE: OTORGADO ]  
Atributo: 'Sexo'  
P(Sexo = 'FEMENINO' | OTORGADO) = 0.4118  
P(Sexo = 'MASCULINO' | OTORGADO) = 0.5882
```

$$\hat{\theta}_{jmk} = \frac{\#\{X_j = x_{jm} \wedge Y = C_k\} + 1}{\#\{Y = C_k\} + M_j}$$

PREDICCIONES

Aplicamos el Teorema de Bayes para calcular la probabilidad a posteriori de cada clase **C_k** sobre un nuevo ejemplo.

El algoritmo evalúa este producto para cada clase **C_k** (2 en nuestro caso) y asigna la clase que obtenga el puntaje más alto.

```
Predicción para el ejemplo ['FEMENINO', 'UNIVERSITARIO', 'HIPOTECA', 'SI']:  
Score(OTORGADO) = 0.000011  
Score(RECHAZADO) = 0.014674
```

$$P(X_1 = x_1, \dots, X_d = x_d \mid Y = C_k) = \prod_{j=1}^d P(X_j = x_j \mid Y = C_k)$$

$$Y \leftarrow \operatorname{argmax}_{\{C_k: k=1, \dots, K\}}$$

MÉTRICAS Y TESTEO DEL MODELO

Mettricas de Naive Bayes en el set de prueba:

Aciertos: 231 / 285

Accuracy en Test: 81.05%

Matriz de Confusión:

Verdaderos Positivos (TP): 37

Verdaderos Negativos (TN): 194

Falsos Positivos (FP): 32

Falsos Negativos (FN): 22

Accuracy: 81.05%

Recall: 62.71%

Especificidad: 85.84%

Precisión: 53.62%

F1-score: 57.81%

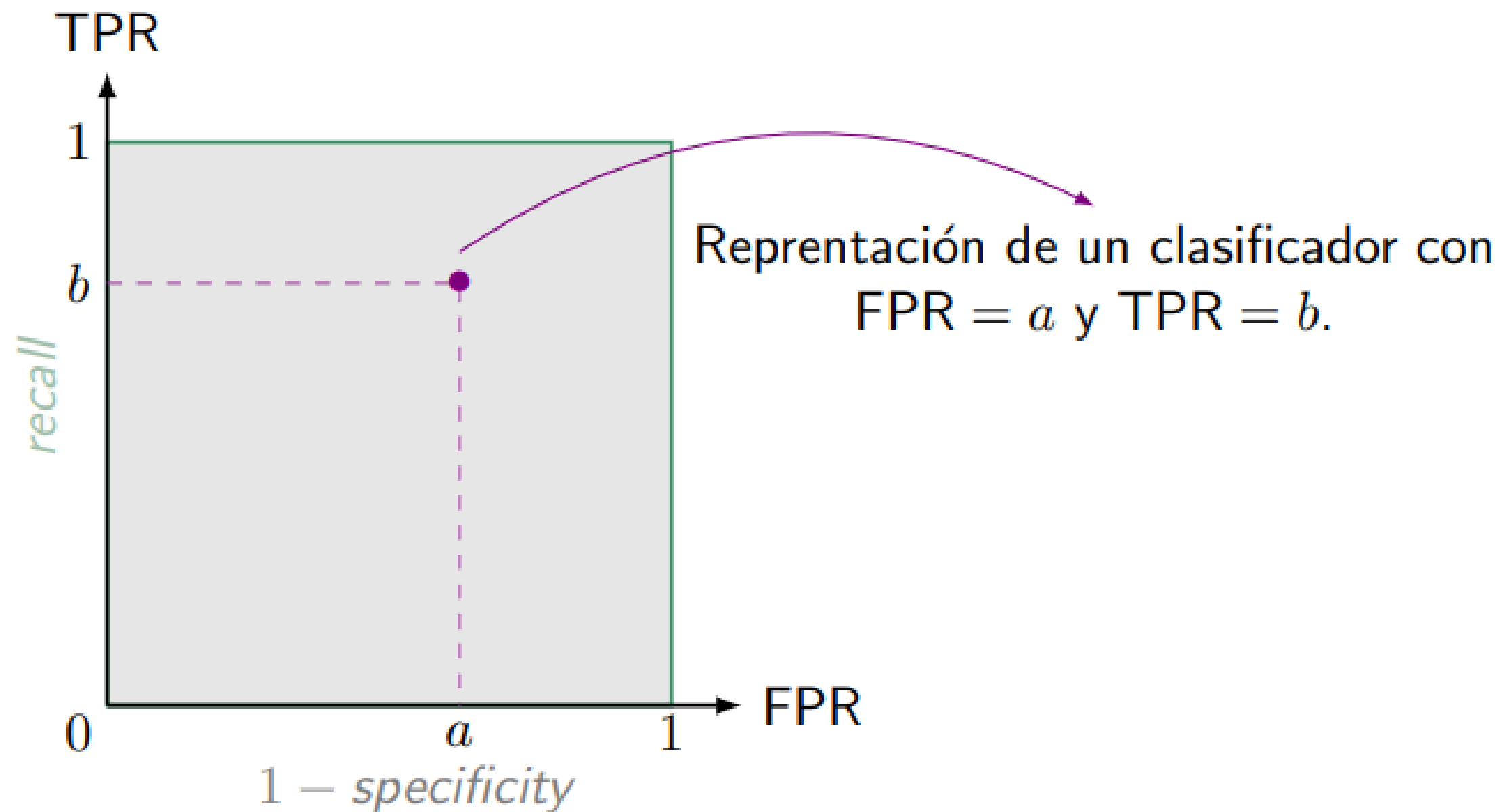
Tasa de Verdaderos Positivos (TPR): 62.71%

Tasa de Falsos Positivos (FPR): 14.16%



ESPACIO ROC

Nos permite representar un clasificador como un punto en un área.
Es el espacio donde vamos a graficar nuestra curva ROC.



Usa un umbral de decisión para categorizar la probabilidad de un ejemplo como de una clase u otra

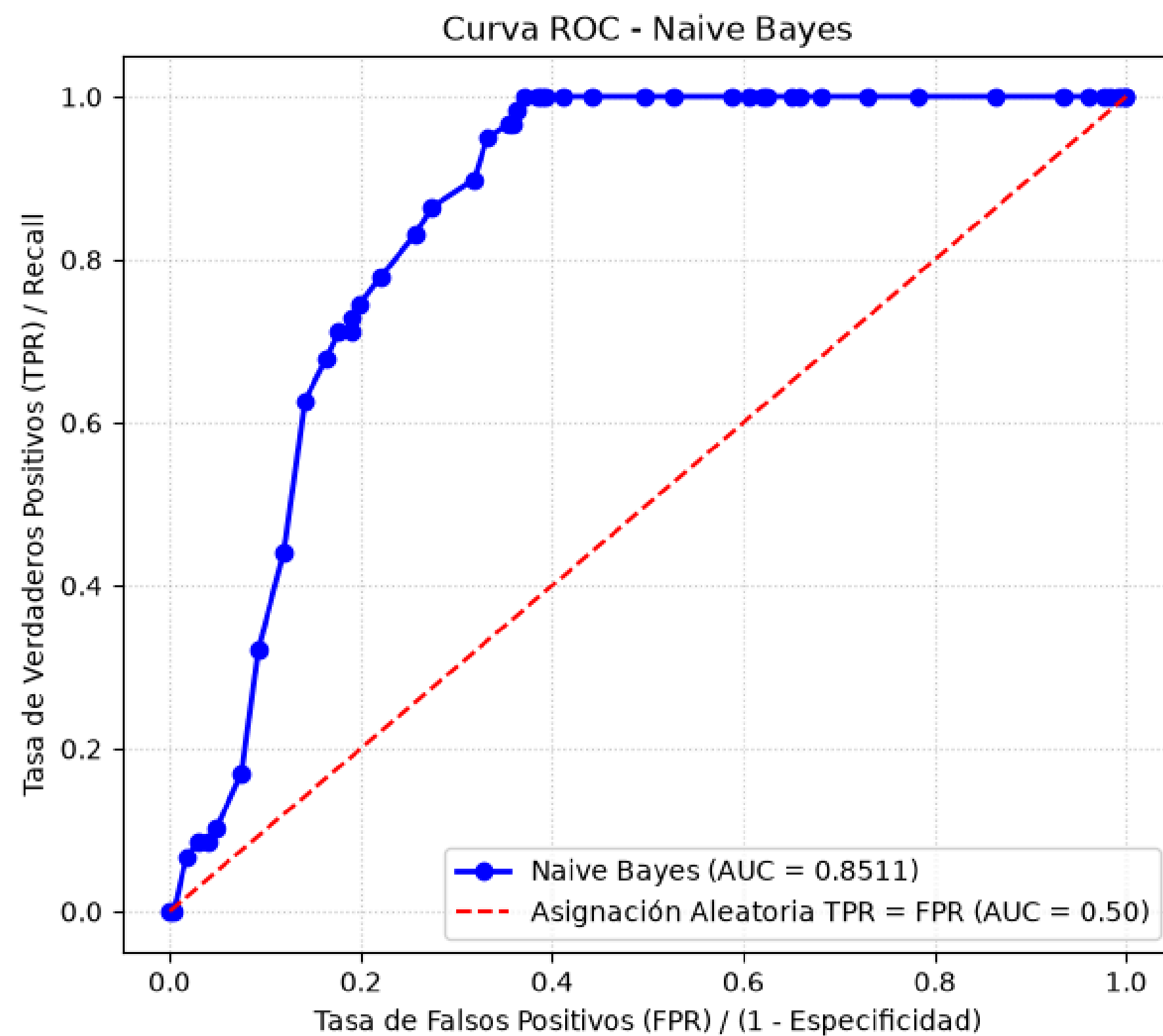
UMBRALES

Para los umbrales, usamos las probabilidades/scores de cada ejemplo, los cuales calculamos anteriormente. Con cada umbral calculamos la matriz de confusion y los puntos en la curva.

```
Umbral: 0.6477 - TP: 0, FP: 1, P: 59, N: 226 - TPR (y): 0.6271, FPR (x): 0.1416
Umbral: 0.5594 - TP: 4, FP: 4, P: 59, N: 226 - TPR (y): 0.0000, FPR (x): 0.0044
Umbral: 0.5566 - TP: 5, FP: 7, P: 59, N: 226 - TPR (y): 0.0678, FPR (x): 0.0177
Umbral: 0.5523 - TP: 5, FP: 9, P: 59, N: 226 - TPR (y): 0.0847, FPR (x): 0.0310
```

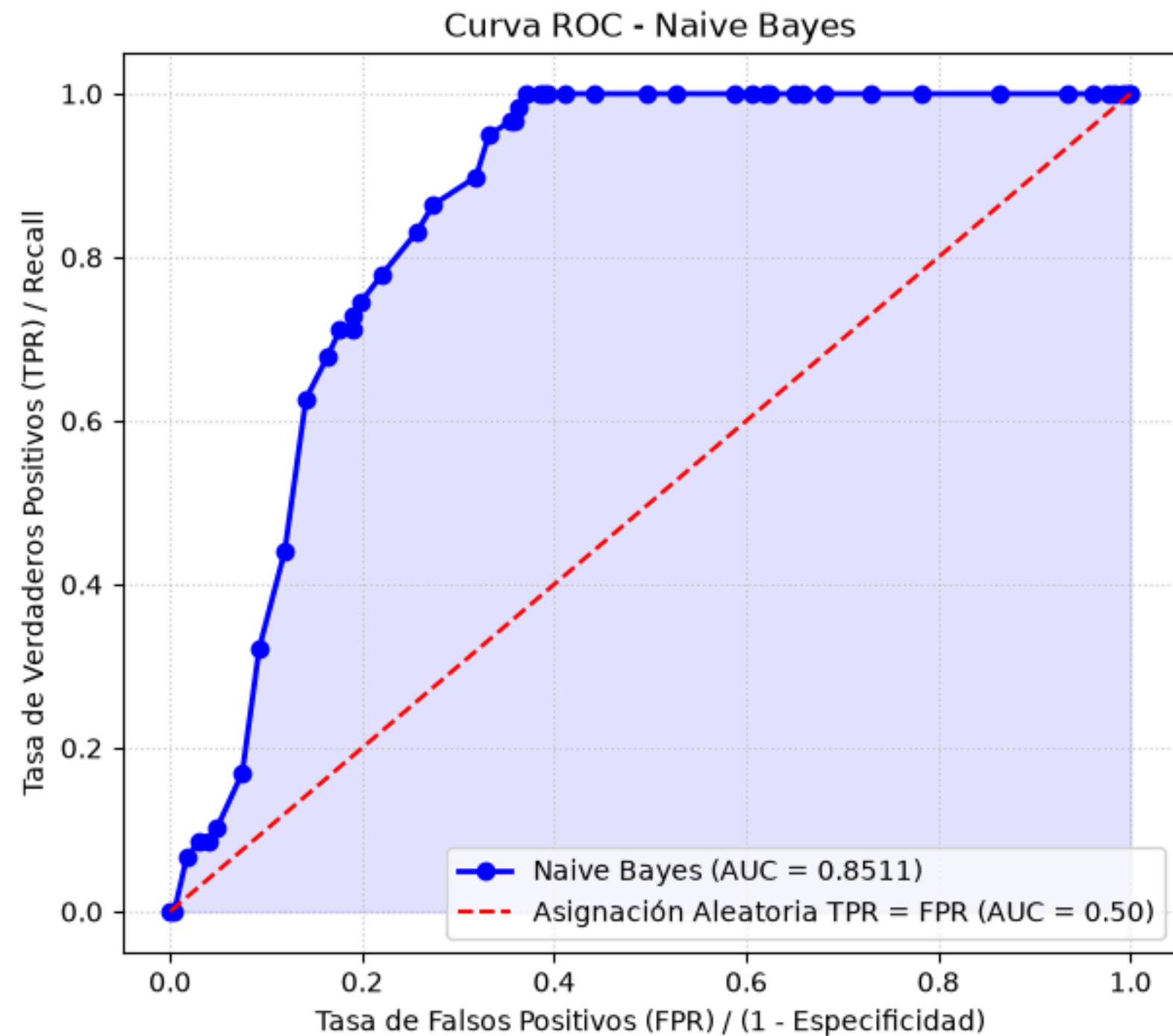
(Tambien hicimos una prueba con umbrales fijos escalonados, pero se pueden observar mejor las transiciones de la forma mencionada arriba)

CURVA ROC



AUC

Es una métrica que se utiliza para comparar el desempeño de diferentes modelos de clasificación.



Como nos dio un valor de aproximadamente 0.85, podemos decir que es un clasificador razonable.

CODE REVIEW

#3

¡GRACIAS!